

DIAGNÓSTICO

CLEAN CODE Y ARQUITECTURA LIMPIA

Mapa conceptual integrador, auditoría de código y defensa técnica individual

PROPÓSITO DE LA ACTIVIDAD

Determinar con evidencia si el estudiante comprende cómo transformar código que “funciona” en software legible, mantenible, testeable, desacoplado y evolutivo. La actividad no evalúa memoria de definiciones aisladas: exige relaciones, decisiones, anti-patrones, ejemplos, refactorización y argumentación técnica.

Campo	Definición
Asignatura / módulo	Clean Code y Arquitectura Limpia
Tipo de actividad	Diagnóstico inicial individual
Duración mínima	120 minutos de trabajo real
Producto principal	Mapa conceptual extenso y evidencia técnica complementaria
Modalidad de defensa	Oral individual, sin lectura mecánica
Lenguaje sugerido	C++, Java, C# o lenguaje definido por el docente

Estudiante: _____ Fecha: _____

Docente: _____ Paralelo: _____

1. Naturaleza del diagnóstico

Este diagnóstico parte de una premisa profesional: un programa puede compilar, ejecutar y producir resultados correctos, y aun así ser un mal producto de ingeniería. Clean Code y Arquitectura Limpia estudian cómo diseñar software que pueda comprenderse, modificarse, probarse, desplegarse y evolucionar sin que cada cambio introduzca miedo, duplicación o dependencia accidental.

REGLA CENTRAL

No se calificará un mapa decorativo. Se calificará la calidad de las relaciones, la precisión de los conceptos, la capacidad de distinguir niveles de diseño y la evidencia de comprensión auténtica. Cada nodo importante debe responder: qué significa, para qué sirve, qué problema evita, cómo se reconoce y cómo se aplica.

2. Objetivos diagnósticos

- Determinar si el estudiante diferencia código funcional de código mantenible.
- Comprobar dominio de nombres, funciones, comentarios, formato, manejo de errores, pruebas y refactorización.
- Identificar si comprende cohesión, acoplamiento, abstracción, encapsulamiento, dependencias y límites arquitectónicos.
- Verificar comprensión de SOLID, separación de responsabilidades y dependencia hacia abstracciones.
- Observar si puede reconocer code smells y proponer mejoras justificadas.
- Comprobar si distingue arquitectura, diseño detallado, patrones, framework e infraestructura.
- Medir su capacidad para explicar decisiones y defender una refactorización sin depender de definiciones memorizadas.

3. Producto obligatorio

El estudiante entregará un mapa conceptual único, jerárquico e interconectado, acompañado por evidencias técnicas. El mapa debe poder leerse como una explicación del sistema completo de ideas y no como una colección de cuadros aislados.

Exigencia	Mínimo verificable
Extensión mínima	90 conceptos o nodos significativos.
Relaciones	Al menos 120 conectores con verbos o frases de enlace.
Ramas principales	Mínimo 14 ramas obligatorias.
Relaciones cruzadas	Mínimo 25 conexiones entre ramas diferentes.
Ejemplos	Mínimo 12 fragmentos breves de código o pseudocódigo.
Anti-ejemplos	Mínimo 10 casos de código deficiente y su mejora.
Decisiones	Mínimo 8 comparaciones justificadas entre alternativas.
Diagnóstico personal	Semáforo de dominio y plan de nivelación.
Defensa	Exposición individual de 5 a 8 minutos más preguntas.

4. Reglas de construcción del mapa

1. El concepto central será “Software limpio, mantenible y evolutivo”.
2. Cada flecha debe contener una frase de enlace: “reduce”, “depende de”, “protege”, “viola”, “facilita”, “se verifica mediante”, “debe permanecer independiente de”, etc.
3. No se aceptan flechas mudas, listados lineales ni definiciones copiadas sin relación.
4. Cada rama debe incluir definición, propósito, señales positivas, señales de riesgo, ejemplo y conexión con otras ramas.
5. Deben diferenciarse claramente decisiones de nivel de código, diseño de clases, componentes y arquitectura.
6. Los ejemplos deben ser pequeños, legibles y explicables por el estudiante.
7. Todo término utilizado debe poder defenderse oralmente con un caso concreto.

5. Arquitectura obligatoria del mapa conceptual

N.º	Rama obligatoria	Cobertura mínima
1	Mentalidad profesional	Legibilidad, mantenibilidad, cambio, costo total, deuda técnica, calidad interna.
2	Nombres significativos	Intención, dominio, consistencia, pronunciabilidad, búsqueda, contexto.
3	Funciones limpias	Tamaño, una responsabilidad, nivel de abstracción, argumentos, efectos secundarios.
4	Comentarios y documentación	Comentarios útiles, ruido, código autoexplicativo, contratos, decisiones.
5	Formato y organización	Jerarquía visual, proximidad, orden, estructura de archivos y módulos.
6	Manejo de errores	Excepciones, códigos, validación, null, precondiciones, mensajes y recuperación.
7	Objetos y estructuras de datos	Encapsulamiento, comportamiento, DTO, Tell Don't Ask, Ley de Demeter.
8	Clases y módulos	Responsabilidad, cohesión, tamaño, invariantes, estabilidad.
9	Code smells y refactorización	Duplicación, funciones largas, clases dios, primitivas obsesivas, cambios divergentes.
10	Pruebas limpias	FIRST, AAA, testabilidad, dobles de prueba, regresión, cobertura con criterio.
11	Principios SOLID	SRP, OCP, LSP, ISP, DIP; intención, violaciones y consecuencias.
12	Dependencias y acoplamiento	Acoplamiento, abstracciones, inversión, inyección, dirección de dependencias.
13	Arquitectura Limpia	Entidades, casos de uso, adaptadores, frameworks, regla de dependencia.
14	Límites y componentes	Boundaries, puertos, adaptadores, interfaces, DTO, mapeo, independencia tecnológica.
15	Patrones y pragmatismo	Patrones como vocabulario, sobreingeniería, YAGNI, KISS, DRY y trade-offs.
16	Evolución y calidad	Observabilidad, seguridad, rendimiento, despliegue, deuda, revisión y mejora continua.

5.1 Mentalidad profesional

- El código se lee muchas más veces de las que se escribe.
- La calidad interna reduce el costo de cambio, defectos y dependencia de personas concretas.
- Distinguir deuda técnica consciente de negligencia técnica.
- Relacionar limpieza con velocidad sostenible, no con perfeccionismo estético.

Pregunta de defensa: ¿Por qué un código que funciona puede ser profesionalmente inaceptable?

5.2 Nombres significativos

- Revelar intención, unidad, alcance y papel del elemento.

- Evitar abreviaturas ambiguas, ruido, nombres genéricos y desinformación.
- Usar vocabulario del dominio y consistencia semántica.
- Comparar nombre corto correcto vs. nombre largo redundante.

Pregunta de defensa: ¿Qué información debe transmitir un nombre sin necesidad de comentarios?

5.3 Funciones limpias

- Hacer una cosa a un solo nivel de abstracción.
- Reducir argumentos y evitar argumentos booleanos confusos.
- Separar consultas de comandos y controlar efectos secundarios.
- Extraer funciones por intención, no solo por cantidad de líneas.

Pregunta de defensa: ¿Cómo sabe que una función tiene más de una responsabilidad?

5.4 Comentarios y documentación

- Explicar por qué, restricciones, riesgos o decisiones no obvias.
- Evitar comentarios que repiten el código, código comentado y documentación desactualizada.
- Diferenciar comentario de API, contrato, README y registro de decisión.
- Mostrar un comentario malo y reemplazarlo con mejor código.

Pregunta de defensa: ¿Qué comentario debería eliminarse y cuál debería conservarse?

5.5 Formato y organización

- Ordenar de lo general a lo específico.
- Mantener conceptos relacionados próximos.
- Definir convenciones consistentes sin convertirlas en ritual.
- Relacionar estructura de carpetas con arquitectura y dominio.

Pregunta de defensa: ¿Cómo influye la organización física en la comprensión del diseño?

5.6 Manejo de errores

- No ocultar fallos ni devolver valores ambiguos.
- Usar excepciones o resultados explícitos según contexto.
- Validar en los límites del sistema.
- Evitar null cuando una abstracción más segura es posible.

Pregunta de defensa: ¿Qué diferencia existe entre reportar un error y ocultarlo?

5.7 Objetos y estructuras de datos

- Diferenciar objetos con comportamiento de estructuras que exponen datos.
- Proteger invariantes mediante encapsulamiento.
- Evitar cadenas de navegación profundas y conocimiento excesivo.
- Explicar cuándo un DTO sí es apropiado.

Pregunta de defensa: ¿Cuándo conviene comportamiento encapsulado y cuándo un DTO?

5.8 Clases y módulos

- Una razón dominante de cambio.
- Alta cohesión y dependencias explícitas.
- Evitar clases dios, utilitarios universales y estados globales.
- Definir invariantes y contratos del módulo.

Pregunta de defensa: ¿Qué significa realmente “una razón para cambiar”?

5.9 Code smells y refactorización

- El smell es una señal, no una sentencia automática.
- Refactorizar sin cambiar comportamiento observable.
- Aplicar cambios pequeños con pruebas de regresión.
- Relacionar smells con causas arquitectónicas.

Pregunta de defensa: ¿Cómo demostraría que una refactorización no alteró el comportamiento?

5.10 Pruebas limpias

- Rápidas, independientes, repetibles, autoevaluables y oportunas.
- Un concepto por prueba y nombres que describen conducta.
- Aplicar Arrange-Act-Assert o Given-When-Then.
- Evitar pruebas frágiles acopladas a implementación.

Pregunta de defensa: ¿Por qué una cobertura alta no garantiza buenas pruebas?

5.11 Principios SOLID

- SRP: razón de cambio y actor responsable.
- OCP: extender comportamiento sin modificar núcleos estables.
- LSP: sustitución sin romper contratos.
- ISP: interfaces pequeñas orientadas al cliente.
- DIP: políticas dependen de abstracciones, no de detalles.

Pregunta de defensa: Explique una violación SOLID y su consecuencia real.

5.12 Dependencias y acoplamiento

- Distinguir dependencia de compilación, ejecución, datos y despliegue.
- Preferir acoplamiento explícito y controlado.
- Usar inversión e inyección para separar política de mecanismo.
- Reconocer cuándo una abstracción accidental empeora el diseño.

Pregunta de defensa: ¿Qué dependencia debería invertirse y por qué?

5.13 Arquitectura Limpia

- Entidades: reglas de negocio de mayor estabilidad.
- Casos de uso: reglas específicas de aplicación.
- Adaptadores: traducción entre formatos y modelos.
- Frameworks y drivers: detalles reemplazables.
- La dependencia del código apunta hacia políticas más internas.

Pregunta de defensa: ¿Qué componentes deberían sobrevivir si cambia la base de datos?

5.14 Límites y componentes

- Un límite protege decisiones de alto nivel.
- Puertos expresan capacidades; adaptadores implementan integración.
- DTO y mappers evitan contaminar el dominio.
- Framework, UI, base de datos y mensajería deben quedar en la periferia.

Pregunta de defensa: ¿Qué información puede cruzar un límite arquitectónico?

5.15 Patrones y pragmatismo

- Un patrón resuelve un problema recurrente en contexto.

- No aplicar patrones por moda o para “demostrar conocimiento”.
- KISS reduce complejidad accidental; YAGNI evita anticipación especulativa.
- DRY elimina conocimiento duplicado, no toda similitud textual.

Pregunta de defensa: ¿Cuándo un patrón se convierte en sobreingeniería?

5.16 Evolución y calidad

- Calidad incluye seguridad, observabilidad, desempeño y operabilidad.
- Revisión de código y análisis estático apoyan, no reemplazan criterio.
- La arquitectura debe facilitar pruebas, cambios y reemplazo de detalles.
- Registrar decisiones y deuda técnica con plan explícito.

Pregunta de defensa: ¿Cómo se relaciona arquitectura con operación y cambio futuro?

6. Evidencias técnicas obligatorias

El mapa se acompañará con una sección práctica. No se aceptará una entrega puramente teórica.

Código	Evidencia	Requisito
A	Nombrado	Presentar 5 nombres deficientes y reemplazarlos justificando intención y contexto.
B	Función larga	Tomar una función de 25 líneas o más y proponer su descomposición.
C	Duplicación	Mostrar duplicación real y explicar qué conocimiento se repite.
D	SOLID	Presentar una violación y una versión mejorada.
E	Dependencia	Dibujar dependencia directa a base de datos/framework y luego invertirla.
F	Prueba	Diseñar 3 pruebas: normal, límite y adversarial.
G	Arquitectura	Clasificar elementos en entidad, caso de uso, adaptador o detalle.
H	Trade-off	Defender una decisión donde “más abstracción” no sea automáticamente mejor.

7. Caso integrador de auditoría

Analice el siguiente escenario: una aplicación registra pedidos. Una única clase recibe datos de interfaz, valida, calcula descuentos, accede directamente a la base de datos, envía correo, genera PDF y registra mensajes en consola. Cada cambio obliga a modificar la misma clase y las pruebas necesitan levantar infraestructura real.

TAREA DE ANÁLISIS

Ubique el caso en su mapa y explique al menos 12 problemas: responsabilidades, acoplamiento, dependencias, testabilidad, manejo de errores, nombres, cohesión, límites, infraestructura, cambios futuros, duplicación y observabilidad. Luego proponga una arquitectura mínima mejorada sin convertirla en una solución sobrediseñada.

8. Identifique actores o razones de cambio diferentes.
9. Defina las reglas de negocio que deberían quedar aisladas.
10. Proponga uno o más casos de uso.
11. Defina puertos necesarios sin mencionar tecnología concreta.
12. Proponga adaptadores para persistencia, correo y documentos.
13. Explique qué datos cruzan cada límite.

14. Diseñe al menos tres pruebas sin infraestructura real.
15. Indique qué simplificaciones mantendría para no sobrearquitectar.

8. Cronograma obligatorio de 120 minutos

Tiempo	Fase	Resultado
0-10	Preparación	Leer consigna, definir medio y organizar espacio.
10-25	Estructura	Crear concepto central, ramas y jerarquía.
25-65	Construcción conceptual	Desarrollar definiciones, relaciones y conexiones cruzadas.
65-90	Evidencia técnica	Incorporar ejemplos, anti-ejemplos y caso integrador.
90-105	Autodiagnóstico	Marcar dominio, vacíos y prioridades.
105-115	Revisión	Contar nodos, relaciones y verificar legibilidad.
115-120	Preparación oral	Seleccionar ruta de explicación y preguntas críticas.

9. Autodiagnóstico obligatorio

Cada rama debe marcarse con uno de los siguientes niveles y una evidencia concreta. No se permite marcar todo como dominado sin justificación.

Nivel	Criterio
Verde - Domino	Puedo explicar, reconocer, aplicar y defender con un ejemplo propio.
Amarillo - Parcial	Reconozco el concepto, pero tengo dudas al aplicarlo o compararlo.
Rojo - Riesgo	No puedo explicarlo con precisión o dependo de definiciones memorizadas.

PLAN PERSONAL

El estudiante seleccionará cinco conceptos en amarillo o rojo y formulará para cada uno: vacío detectado, acción de estudio, ejercicio práctico, evidencia esperada y fecha de recuperación.

10. Defensa oral individual

16. Explicar el concepto central y la lógica global del mapa en un minuto.
17. Recorrer dos ramas elegidas por el docente, no por el estudiante.
18. Explicar una relación cruzada entre Clean Code y Arquitectura Limpia.
19. Defender una refactorización y el riesgo que reduce.
20. Responder una variante o cambio de requisito.
21. Distinguir mejora real de sobreingeniería.
22. Explicar una decisión que no tenga una única respuesta correcta.

11. Banco de preguntas para la defensa

- ¿Cuál es la diferencia entre código limpio, buen diseño y buena arquitectura?
- ¿Puede una función corta seguir teniendo demasiadas responsabilidades?

- ¿Cuándo DRY puede producir una abstracción incorrecta?
- ¿Qué problema real resuelve DIP?
- ¿Cuál es la diferencia entre inyección de dependencias e inversión de dependencias?
- ¿Qué parte de una aplicación debería contener las reglas de negocio?
- ¿Qué sucede si las entidades conocen detalles de base de datos?
- ¿Cómo detecta una violación de LSP?
- ¿Por qué una interfaz grande perjudica a sus clientes?
- ¿Cuándo un comentario es una señal de mal código?
- ¿Cómo prueba un caso de uso sin base de datos real?
- ¿Qué es un límite arquitectónico y qué protege?
- ¿Qué puede cruzar un límite sin contaminar el dominio?
- ¿Por qué “usar muchas capas” no garantiza Arquitectura Limpia?
- ¿Cuándo una clase utilitaria es una señal de diseño pobre?
- ¿Qué relación existe entre deuda técnica y costo de cambio?
- ¿Cómo distinguir complejidad esencial de complejidad accidental?
- ¿Qué refactorización aplicaría primero y por qué?
- ¿Qué evidencia demuestra que una mejora no rompió comportamiento?
- ¿Qué parte de su mapa considera más débil y cómo la reforzará?

12. Rúbrica de evaluación

Criterio	Desempeño esperado	Peso
Cobertura conceptual	Incluye y desarrolla las 16 ramas con precisión.	20%
Calidad de relaciones	Conectores significativos, jerarquía y relaciones cruzadas.	20%
Aplicación técnica	Ejemplos, anti-ejemplos, refactorización y caso integrador.	20%
Comprensión arquitectónica	Distingue políticas, mecanismos, límites y dependencias.	15%
Pensamiento crítico	Trade-offs, pragmatismo y ausencia de dogmatismo.	10%
Presentación y legibilidad	Orden, coherencia visual y trazabilidad.	5%
Defensa oral	Explica, justifica, responde variantes y reconoce límites.	10%

13. Niveles de desempeño

Nivel	Rango	Descripción
Excelente	90-100	Relaciona principios, detecta problemas, propone mejoras justificadas y evita dogmatismo.
Competente	75-89	Comprende y aplica la mayoría de los conceptos con errores menores.
En desarrollo	60-74	Reconoce términos, pero presenta relaciones débiles o aplicaciones mecánicas.
Insuficiente	0-59	Mapa superficial, definiciones aisladas, ejemplos copiados o incapacidad de defensa.

14. Indicadores de riesgo académico

- Confunde Clean Code con formato o gustos personales.

- Recita SOLID sin reconocer violaciones en código.
- Afirma que toda función debe tener un número fijo de líneas.
- Usa patrones, interfaces o capas sin problema concreto que resolver.
- No distingue reglas de negocio de detalles tecnológicos.
- Necesita una base de datos o framework para probar cualquier comportamiento.
- No puede explicar por qué una dependencia apunta en determinada dirección.
- Considera que cobertura de pruebas equivale a calidad.
- Propone refactorizaciones masivas sin protección de pruebas.
- No reconoce trade-offs ni costos de abstracción.

15. Lista de verificación del estudiante

- ☐ Mi mapa tiene un concepto central claro y al menos 16 ramas.
- ☐ Incluí 90 conceptos y 120 relaciones significativas como mínimo.
- ☐ Todas las flechas importantes tienen frases de enlace.
- ☐ Incluí 25 relaciones cruzadas entre ramas.
- ☐ Añadí ejemplos y anti-ejemplos que puedo explicar.
- ☐ Diferencí código, diseño, componentes y arquitectura.
- ☐ Analicé el caso integrador y propuse una mejora mínima.
- ☐ Identifiqué trade-offs y riesgos de sobreingeniería.
- ☐ Marqué mi dominio con verde, amarillo o rojo.
- ☐ Preparé cinco acciones concretas de nivelación.
- ☐ Puedo defender el mapa sin leerlo mecánicamente.
- ☐ Puedo responder una modificación o requisito inesperado.

16. Formato de registro docente

Dimensión	Observación	Puntaje / nivel
Cobertura conceptual		
Relaciones y jerarquía		
Ejemplos técnicos		
Comprensión SOLID		
Comprensión arquitectónica		
Pragmatismo y trade-offs		
Defensa oral		
Nivel global		
Vacíos prioritarios		
Acción de nivelación		

17. Cierre pedagógico

El diagnóstico habrá cumplido su función cuando el estudiante deje de preguntar únicamente “¿funciona?” y comience a preguntar “¿se entiende?, ¿puede probarse?, ¿qué responsabilidad tiene?, ¿qué dependencia introduce?, ¿qué cambio futuro lo afectará?, ¿qué política debe quedar protegida?, ¿qué evidencia demuestra que la mejora es real?”. Ese cambio de criterio marca el paso desde programar instrucciones hacia diseñar software profesional.

MENSAJE FINAL PARA EL ESTUDIANTE

La limpieza no es decoración y la arquitectura no es una colección de carpetas. Ambas son disciplinas para controlar el cambio, proteger decisiones importantes y mantener el software comprensible. La calidad se demuestra con decisiones justificadas, pruebas y capacidad de evolución.